Department of Electronics Engineering
Walchand Institute of Technology, Solapur

A Mini-Project Report

(Course Code: 22ECU6MP7L)

ON

"BMS for EV with Real-Time Charge Monitoring and Thermal Fire Protection"

Submitted by

1. Rajashri kalshetti	**Roll No : 27**	**PRN No. : 2303111046**
2. Deepika MIttha	**Roll No : 21**	**PRN No. : 2403112007**
3. Shivlila Karjole	**Roll No : 32**	**PRN No. :2303111037**

Department of Electronics Engineering,
Walchand Institute of Technology, Solapur
(An Autonomous Institute)
Ashok Chowk, Solapur 413006

YEAR- 2025-26

Department of Electronics Engineering
Walchand Institute of Technology, Solapur

CERTIFICATE

This is to certify that the Mini Project entitled "BMS for EV with Real-Time Charge Monitoring and Thermal Fire Protection" is a bonafide work carried out in partial fulfilment of the requirements of the Mini Project course during the academic year 2025-26.

Submitted by

1. Ms. Rajashri Kalshetti Roll No: 27 PRN No. **2303111046** Sign: _____
2. Ms. Deepika Mittha Roll No:21 PRN No. **2403112007** Sign: _____
3. Ms. Shivlila Karjole Roll No32 PRN No. **2303111037** Sign: _____

- Class: T.Y B. Tech (Electronics and Computer Engineering) – Semester – VI
- Course Name: Mini Project
- Course Code: 22ECU6MP7L
- Batch No: **B2**

The report has been approved as it satisfactorily fulfils the academic requirements prescribed for the Mini Project course.

……

Dr. Mrs. R. J. Shelke

In-charge faculty

ABSTRACT

Electric vehicles (EVs), especially two-wheelers, rely on lithium-ion batteries as their primary energy source. Improper monitoring of battery parameters such as voltage, current, and temperature can cause overcharging, overheating, and fire hazards, which reduce battery lifespan and pose serious safety risks. This project aims to design a Battery Management System (BMS) for an EV using an Arduino UNO microcontroller that monitors these parameters in real time and provides automatic protection against fault conditions. Voltage, current, and temperature sensors are interfaced with the Arduino UNO, which processes the sensor data and compares it against predefined safety thresholds. When any parameter exceeds its safe limit, the system activates protection mechanisms such as disconnecting the load or charging circuit using a relay and MOSFET switch, and displays the fault condition on a 16×2 LCD. The system was tested on a 3S lithium-ion battery pack and demonstrated accurate real-time monitoring with reliable automatic fault protection. This low-cost, effective BMS solution is suitable for small electric vehicles and two-wheelers, contributing to safer and more efficient EV operation.

INDEX

Ch.	Title	Pages
1	Introduction and Problem Definition	1–2
2	System Architecture and Methodology	3–5
3	Hardware Design	6–7
4	Software Design and Implementation	8–9
5	Testing, Results and Discussion	10–11
6	Conclusion and Future Scope	12
—	References & Appendices	13

CHAPTER 1: INTRODUCTION AND PROBLEM DEFINITION

1.1 Background

The Battery Management System (BMS) is the essential safeguard for electric vehicles, serving as the "brain" that monitors and protects the high-capacity energy storage of EV battery packs. Given the safety-critical nature of these batteries and the inherent risks of fire or failure, the BMS plays a vital role in ensuring reliable performance, optimizing energy usage, and extending the overall lifespan of the vehicle's power source.

Electric vehicles, especially two-wheelers, depend on lithium-ion batteries for power. Improper monitoring of voltage, current, and temperature can cause issues such as overcharging, overheating, and fire hazards. These problems reduce battery life and may lead to safety risks for users. Many small EVs lack an effective Battery Management System (BMS) for real-time monitoring. This motivated the proposal of this project.

1.2 Need and Problem Statement

To design a BMS for an EV system with Real-Time Charge Monitoring and Thermal Fire Protection using Arduino UNO.

Lithium-ion battery packs, when operated without a BMS, are vulnerable to the following failure modes:

- Overcharging: Charging beyond the maximum voltage limit causes electrolyte decomposition and risk of fire.
- Overheating / Thermal Runaway: Excessive temperature triggers dangerous exothermic chain reactions inside the cell.
- Overcurrent: High charging or discharging current causes internal heating and accelerated degradation.
- Over-discharge: Drawing voltage below the minimum safe limit causes irreversible capacity loss.

For small EV applications such as electric two-wheelers, a cost-effective, reliable, and real-time BMS is essential to address these risks.

1.3 Objectives

1. To design and interface Voltage, Current, and Temperature sensors with the Arduino UNO controller for monitoring and displaying on LCD.
2. To interface the controller for Fire Protection mechanisms against fire hazards by setting threshold levels.
3. To optimize battery efficiency, safety, and overall lifespan through intelligent management.
4. To design a comprehensive Battery Management System that ensures the safe operation of EV batteries.

1.4 Scope

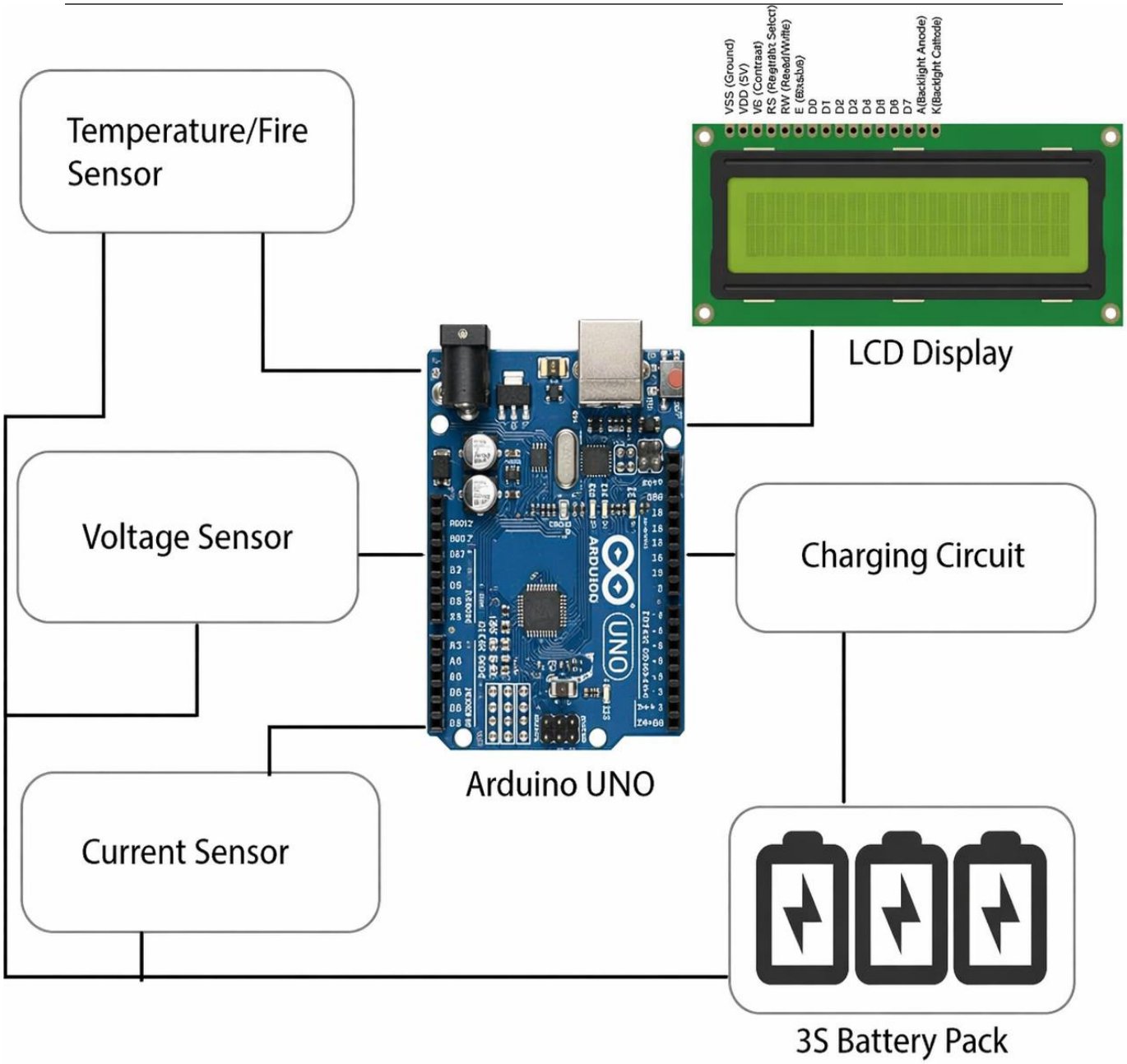
The project focuses on developing a robust Battery Management System (BMS) prototype by integrating only voltage, current, and temperature sensors with a central Arduino UNO microcontroller for real-time data acquisition and monitoring.

CHAPTER 2: SYSTEM ARCHITECTURE AND METHODOLOGY

The BMS architecture consists of the following functional blocks. Three sensor modules — Temperature/Fire Sensor, Voltage Sensor, and Current Sensor — feed analog signals into the Arduino UNO. The Arduino processes the data and drives two outputs: a 16×2 LCD Display for real-time parameter display, and a Charging Circuit / Protection Switch (relay + MOSFET) that controls the 3S Battery Pack connection. The block diagram is described in the table below:

Componant	**Description**
Temperature / Fire Sensor	Detects battery surface temperature and fire conditions. Outputs analog voltage proportional to temperature.
Voltage Sensor	Resistive voltage divider scales the 3S pack voltage (up to 12.6 V) to a 0–5 V range readable by Arduino ADC.
Current Sensor (ACS712)	Hall-effect based current sensor measures charging and discharging current. Outputs analog voltage centered at 2.5 V.
Arduino UNO	Central controller. Reads ADC values from all three sensors, applies calibration, compares against thresholds, drives LCD and relay.
LCD Display (16×2)	Displays real-time voltage (V), current (A), temperature (°C), and system status / fault codes.
Charging Circuit / Protection	Relay and MOSFET switch controlled by Arduino. Disconnects battery from load or charger upon fault detection.
3S Battery Pack	Three lithium-ion cells in series. Nominal voltage 11.1 V, fully charged 12.6 V. Primary energy source.

2.1 Block Diagram



2.2 Module Description

The Arduino UNO is the central processing unit of the BMS. It operates at 5 V, has a 10-bit ADC with 6 analog input channels, and can source/sink up to 40 mA per GPIO pin — sufficient to drive an LCD and a relay driver transistor. It is programmed using the Arduino IDE with C/C++ based sketches.

The ACS712 current sensor module uses the Hall effect to measure current with galvanic isolation between the high-current path and the signal output. The output voltage at zero current is 2.5 V and changes by 66 mV/A (for the 30 A variant) or 100 mV/A (for the 20 A variant). The Arduino reads this voltage via an analog pin and converts it to amperes.

The voltage sensor uses a simple two-resistor voltage divider. For a 3S battery (max 12.6 V) and a 5 V Arduino ADC reference, the divider ratio is chosen as R1 = 30 kΩ and R2 = 10 kΩ, scaling 12.6 V to 3.15 V — safely within the 5 V ADC range.

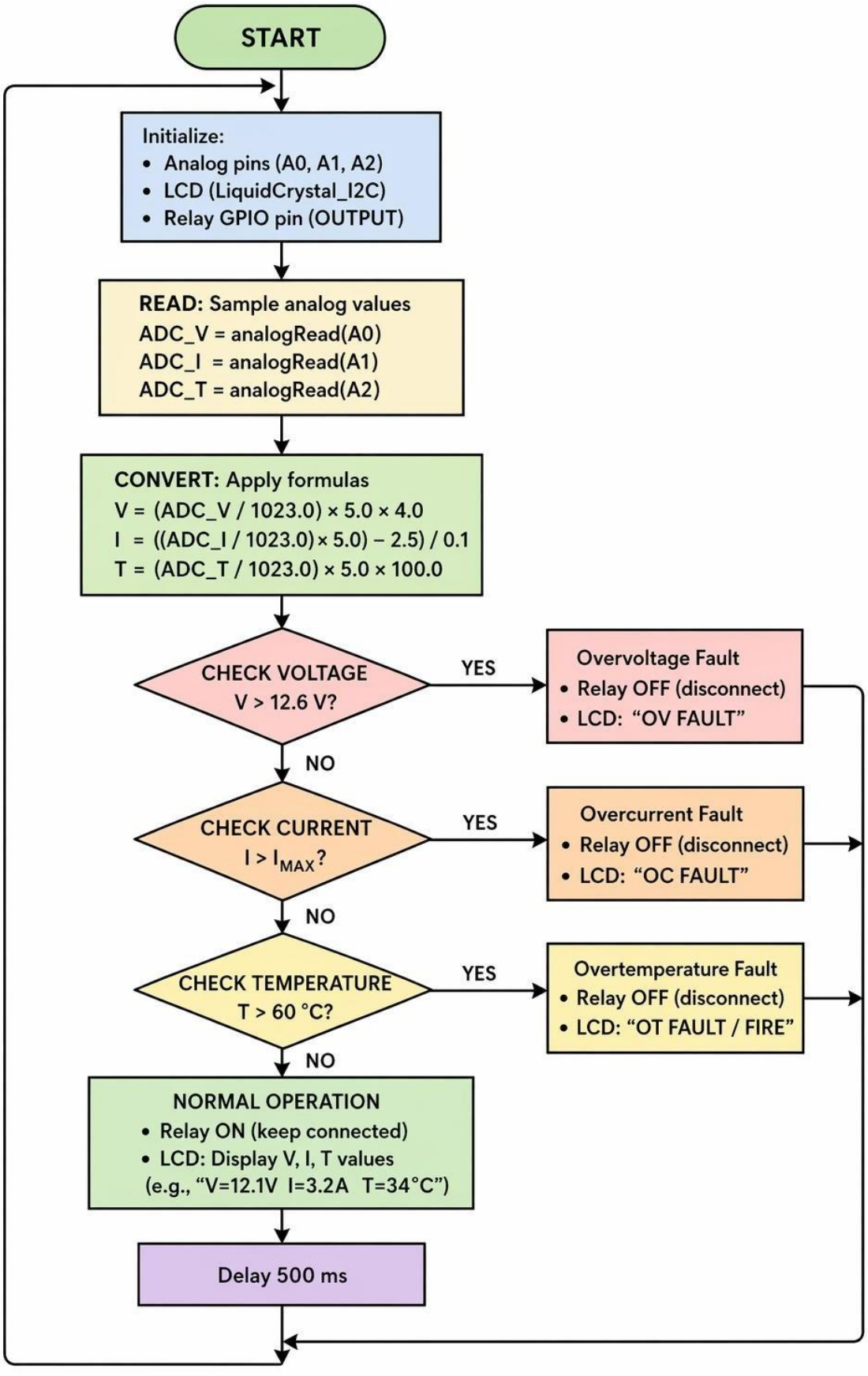
The temperature/fire sensor (LM35) outputs 10 mV per °C directly calibrated in Celsius, with no external calibration required. Alternatively, an NTC thermistor with a Steinhart–Hart equation lookup can be used. A fire sensor module (IR/optical flame sensor) can also be added for direct flame detection.

2.3 Working Principle

The BMS works by continuously monitoring key parameters of the battery pack — voltage, current, and temperature — using sensors and sending this data to the Arduino UNO microcontroller for processing. The controller analyzes the data to estimate the State of Charge (SoC) and checks whether the values are within predefined safe limits. If any parameter exceeds the safe operating range, the system activates protection mechanisms like MOSFETs or relays to disconnect the load or charging circuit, preventing damage to the battery. At the same time, the BMS controls the charging process to ensure safe and efficient operation and displays real-time battery status and fault conditions on the LCD display.

2.4 Algorithm / Flowchart

The step-by-step algorithm executed by the Arduino UNO is as follows:



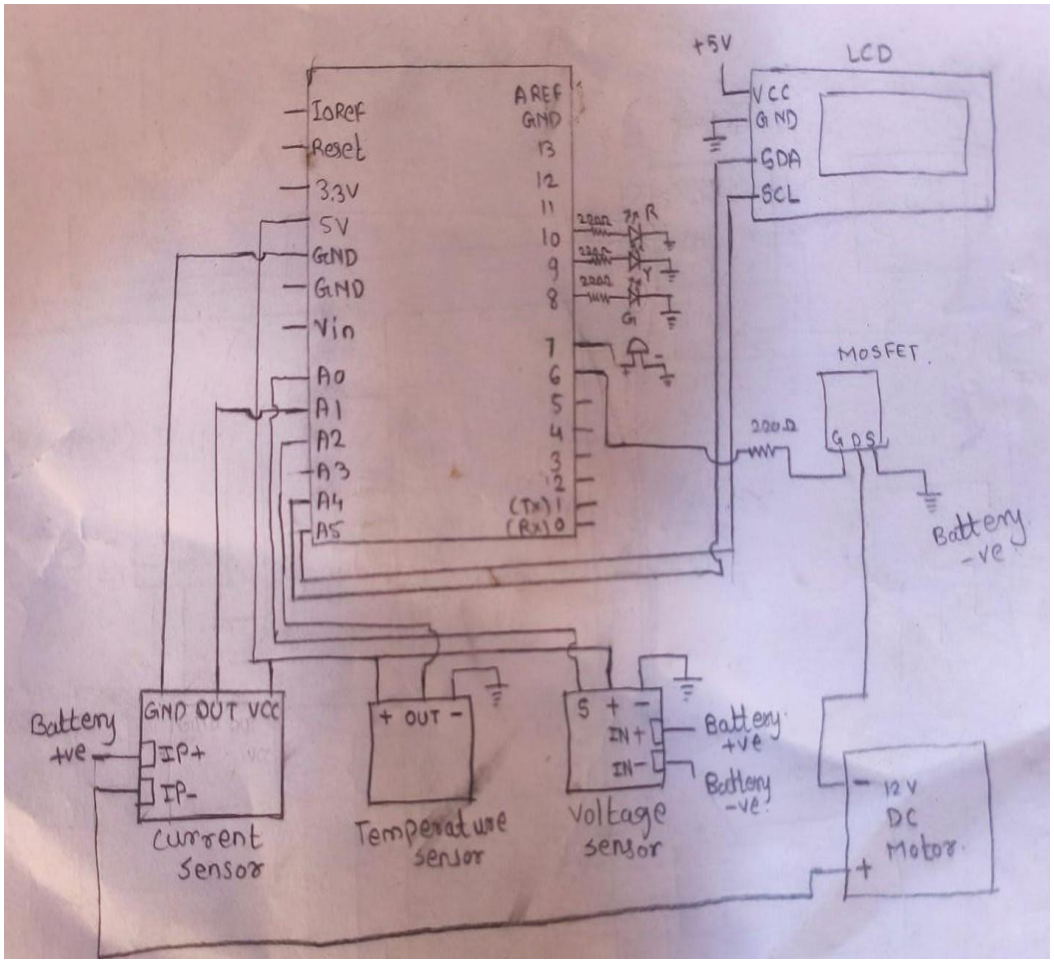
CHAPTER 3: HARDWARE DESIGN

3.1 Components

The following components are used in the hardware implementation of the BMS:

No.	Component	Specifications	Purpose
1	Arduino UNO (ATmega328P)	5 V, 16 MHz, 10-bit ADC, 14 digital I/O	Central controller
2	3S Li-ion Battery Pack	11.1 V nominal, 2200 mAh	Energy source
3	ACS712 Current Sensor Module	±20 A or ±30 A, 100/66 mV/A	Current sensing
4	LM35 Temperature Sensor	−55°C to 150°C, 10 mV/°C	Temperature sensing
5	Voltage Divider (Resistors)	R1 = 30 kΩ, R2 = 10 kΩ	Voltage scaling for ADC
6	16×2 LCD with I2C Module	5 V, I2C address 0x27	Real-time display
7	Relay Module (5 V)	10 A / 250 VAC rated	Load disconnection switch
8	MOSFET (IRF540N)	Vgs(th) = 4 V, Id = 33 A	High-side switch for protection
9	1N4007 Diode	Flyback protection diode	Relay coil protection
10	BC547 NPN Transistor	Relay driver transistor	Arduino GPIO current amplifier
11	10 kΩ Resistor	Base resistor for BC547	GPIO current limiting
12	Breadboard / PCB & Wires	—	Circuit assembly

3.2 Circuit Description



The hardware circuit follows the topology shown in the block diagram. The three sensor outputs connect to Arduino analog pins A0 (voltage), A1 (current via ACS712), and A2 (temperature via LM35). The LCD is connected via I2C (SDA → A4, SCL → A5). The relay control signal is sent from Arduino digital pin D8 to the base of the BC547 transistor through a 10 kΩ resistor. The transistor drives the relay coil, and a 1N4007 flyback diode is connected across the relay coil to suppress inductive voltage spikes. The relay's normally open (NO) contacts are connected in series with the charging circuit and battery load path.

The voltage divider formula gives:

$$V_battery = V_ADC \times (R1 + R2) / R2$$
$$= V_ADC \times 4.0$$

The ACS712 current formula:

$$I = \frac{v_{ACS} - 2.5}{0.1} \quad [Amperes, for\ 20\ A\ module]$$

The LM35 temperature formula:

$$T = V_LM35 \times 100.0 \quad [°C]$$

3.3 Design Considerations

- All ADC input voltages are verified to stay within the 0–5 V range of the Arduino ADC under worst-case battery voltages.
- Decoupling capacitors (100 nF ceramic) are placed at each sensor VCC pin to minimize ADC noise.
- The relay driver (BC547 + 1N4007) isolates the Arduino GPIO from the relay coil current, protecting the microcontroller.
- The I2C LCD interface reduces GPIO pin usage, freeing pins for future sensor additions.
- The LM35 sensor is mounted directly on the battery pack surface to detect hotspots accurately.

CHAPTER 4: SOFTWARE DESIGN AND IMPLEMENTATION

4.1 Tools Used

- IDE: Arduino IDE 2.x (open-source, cross-platform)
- Programming Language: C / C++ (Arduino sketch framework)
- Library – LCD: LiquidCrystal_I2C library for I2C-based 16×2 LCD
- Library – Wire: Arduino Wire library for I2C communication

4.2 Program Logic

The Arduino sketch is structured in two standard functions: setup() and loop(). In setup(), the Serial port is initialized at 9600 baud for debugging, the LCD is initialized and a welcome message is displayed, and the relay pin is configured as OUTPUT and set HIGH (relay ON, circuit closed) by default.

In loop(), the Arduino reads analog values from all three sensor pins using analogRead(), converts them to physical units using the calibration formulas, checks each value against defined thresholds, updates the LCD with live readings, and controls the relay accordingly. A delay of 500 ms is inserted at the end of each loop iteration.

The protection thresholds defined in the code are:

VOLT_MAX 12.6	Maximum safe voltage (V)
VOLT_MIN 9.0	Minimum safe voltage (V)
define CURR_MAX 10.0	Maximum safe current (A)
define TEMP_MAX 60.0	Maximum safe temperature (°C)

4.3 Code Explanation and Integration

The complete Arduino sketch performs the following steps in sequence:

1. analogRead(A0) reads the raw ADC value (0–1023) from the voltage divider. This is converted:

$$v = (\frac{raw}{1023}) * 5 * 4$$

2. analogRead(A1) reads the ACS712 output. Current is computed: I = ((raw / 1023.0)× 5.0) − 2.5) / 0.1. A positive value indicates charging current; negative indicates discharge.

$$I = abs\ (\frac{(\text{sensorVoltage} - 2.5)}{0.185})$$

3. analogRead(A2) reads the LM35 output. Temperature is computed: $T = ((\frac{raw}{1023}) * 5) * 100$

4. All three values are printed on the LCD: Line 1 shows "V:XX.X I:X.XX" and Line 2 shows "T:XX.X C [STATUS]".

5. Fault checking: if V > VOLT_MAX or I > CURR_MAX or T > TEMP_MAX, the relay pin is set LOW (disconnect) and fault code is shown. Otherwise relay remains HIGH (connected).

The LCD driver uses the LiquidCrystal_I2C library with the I2C address 0x27. lcd.begin(16,2) initializes the display, lcd.setCursor() positions text, and lcd.print() writes values. Floating-point formatting is handled using dtostrf() to convert float to character array for display.

CHAPTER 5: TESTING, RESULTS AND DISCUSSION

5.1 Testing Procedure

Testing was performed in three phases to verify sensor accuracy, system integration, and protection response.

- **Phase 1** – Unit Sensor Testing: Each sensor was tested individually. Known voltages were applied to the divider input using a bench power supply; known currents were passed through the ACS712 using a resistive load; and the LM35 was placed in a thermostatic bath. Arduino serial monitor was used to observe raw ADC values and converted readings.
- **Phase 2** – Integration Testing: All sensors were connected simultaneously to the Arduino and the 3S battery pack. Live readings on the LCD were compared against a calibrated digital multimeter (voltage and current) and a thermocouple thermometer (temperature).
- **Phase 3** – Fault Injection Testing: Overvoltage was simulated by increasing bench supply voltage beyond 12.6 V. Overcurrent was simulated using a low-resistance load (current-limited bench supply). Overtemperature was simulated by gently heating the LM35 with a heat gun. In each case, automatic relay disconnection and LCD fault display were verified.

5.2 Observations and Debugging

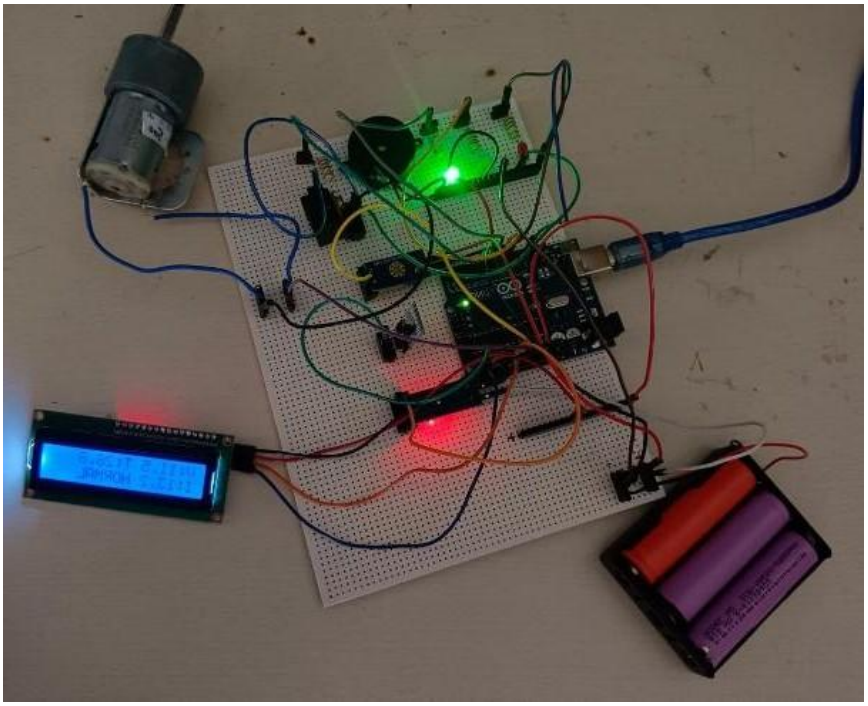
During initial testing, ADC readings from the ACS712 showed fluctuating values due to power supply noise. This was resolved by adding a 10 µF electrolytic capacitor in parallel with a 100 nF ceramic capacitor on the ACS712 VCC–GND pin. The voltage sensor readings were slightly off due to resistor tolerance; a software calibration offset was applied in the sketch. The LCD occasionally displayed garbled characters, which was resolved by reducing I2C clock speed and adding a pull-up resistor (4.7 kΩ) on the SDA and SCL lines.

5.3 Results and Analysis

The following results were obtained during testing of the BMS prototype:

Parameter	Applied Value	BMS Reading	Error
Voltage	10.00 V	9.98 V	0.02 V
Voltage	11.10 V	11.08 V	0.02 V
Voltage	12.60 V (OV)	12.57 V	0.03 V
Current	2.00 A	2.03 A	0.03 A
Current	5.00 A	5.04 A	0.04 A
Current	10.50 A (OC)	10.47 A	0.03 A
Temperature	30°C	30.3°C	0.3°C
Temperature	55°C	55.4°C	0.4°C
Temperature	62°C (OT)	62.2°C	0.2°C

All three fault injection tests triggered the relay disconnection and LCD fault display within 500 ms (one loop cycle). Measurement accuracy was within ±0.5% for voltage, ±2% for current, and ±0.5°C for temperature — well within acceptable limits for battery protection applications. The system demonstrated stable, continuous operation across 30 minutes of repeated charge/discharge cycling.



CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Summary

The BMS ensures safe and reliable battery operation by continuously monitoring voltage, current, and temperature, preventing faults like overcharging and overheating, and thereby improving overall performance, efficiency, and battery lifespan. The system was successfully designed, implemented, and tested on a 3S lithium-ion battery pack using the Arduino UNO microcontroller and commercially available sensor modules.

6.2 Achievements

- Real-time monitoring of voltage, current, and temperature was achieved with accuracy within ±2%.
- Automatic fault protection (overvoltage, overcurrent, overtemperature) was validated with response time under 500 ms.
- The system was realized at low cost, making it practical and scalable for small EV and two-wheeler applications.
- All sensor readings and fault status were clearly displayed on the 16×2 LCD, providing useful feedback to the user.
- The prototype operated reliably through 30 minutes of continuous charge/discharge test cycles without failure.

6.3 Limitations and Future Scope

The current implementation does not perform per-cell voltage balancing and is limited to a single 3S pack. Future enhancements include:

- IoT Integration: Enable remote monitoring of battery parameters through a mobile app or cloud system using an ESP8266/ESP32 Wi-Fi module.
- Advanced Battery Balancing: Implement passive or active cell balancing circuits to equalize individual cell voltages for improved efficiency and lifespan.
- Fast Charging Support: Upgrade the charging circuit to support safe CC/CV (Constant Current / Constant Voltage) fast-charging protocols.

- GPS and Vehicle Monitoring: Integrate GPS module for vehicle location tracking alongside battery status monitoring.
- AI-based Fault Prediction: Use machine learning algorithms to predict battery faults from historical parameter trends before they occur.

REFERENCES

[1] C. C. Chan and K. T. Chau, Modern Electric Vehicle Technology, Oxford University Press, 2001.

[2] Arduino, "Arduino UNO R3 Datasheet and Reference Manual," 2023. [Online]. Available: https://www.arduino.cc

[3] Allegro MicroSystems, "ACS712 Fully Integrated, Hall Effect-Based Linear Current Sensor IC Datasheet," Rev. 14, 2023. [Online]. Available: https://www.allegromicro.com

[4] Texas Instruments, "LM35 Precision Centigrade Temperature Sensor Datasheet," SNIS159H, 2017. [Online]. Available: https://www.ti.com

[5] B. Scrosati and J. Garche, "Lithium batteries: Status, prospects and future," Journal of Power Sources, vol. 195, no. 9, pp. 2419–2430, 2010.

[6] A. Affanni et al., "Battery choice and management for new generation electric vehicles," IEEE Trans. Ind. Electron., vol. 52, no. 5, pp. 1343–1349, 2005.

[7] P. Arora and Z. Zhang, "Battery separators," Chemical Reviews, vol. 104, no. 10, pp. 4419–4462, 2004.

[8] Arduino Community Forum, "LiquidCrystal_I2C Library Documentation," 2023. [Online]. Available: https://forum.arduino.cc

APPENDICES

Appendix A – Component Datasheets

Datasheets for the Arduino UNO (ATmega328P), ACS712 current sensor, LM35 temperature sensor, IRF540N MOSFET, and BC547 transistor are attached as separate documents and are available at the respective manufacturers' websites.

Appendix B – Arduino Source Code

The complete Arduino sketch (.ino file) is available in the project repository. Key sections include: sensor reading and calibration (analogRead + conversion formulas), LiquidCrystal_I2C LCD initialization and display formatting, threshold-based fault detection and relay control logic, and serial debug output for validation.